

Krzysztof Mielnik

Kompilacja jądra Linux

Maj 2025

# 1 Wstęp

Zadanie zostało wykonane na Slackware 15 (tym samym systemie co na zajęciach) na wersji jądra 5.15.207.

```
Last failed login: Sun May 24 10:57:32 CEST 2026 from 192.168.122.1 on ssh:notty
There was 1 failed login attempt since the last successful login.
Last login: Sun May 24 10:43:03 2026 from 192.168.122.1
Linux 5.15.207.
root@slack64:~#
```

Rysunek 1: Wersja systemu

## 2 Stara metoda

### 2.1 Przebieg kompilacji

Najpierw pobrałem źródła najnowszej wersji jądra tj. 7.0.10. Skompresowany plik pobrałem do katalogu `/usr/src`.

```
root@slack64:~# cd /usr/src
root@slack64:/usr/src# wget https://cdn.kernel.org/pub/linux/kernel/v7.x/linux-7.0.10.tar.xz
--2026-05-24 15:06:30-- https://cdn.kernel.org/pub/linux/kernel/v7.x/linux-7.0.10.tar.xz
Translacja cdn.kernel.org (cdn.kernel.org)... 146.75.121.176, 2a04:4e42:8e::432
Łączenie się z cdn.kernel.org (cdn.kernel.org)[146.75.121.176]:443... połączono.
Żądanie HTTP wysłano, oczekiwanie na odpowiedź... 200 OK
Długość: 157261076 (150M) [application/x-xz]
Zapis do: 'linux-7.0.10.tar.xz'

linux-7.0.10.tar.xz          100%[=====>] 149,98M  17,7MB/s   w 8,4s
2026-05-24 15:06:38 (17,8 MB/s) - zapisano 'linux-7.0.10.tar.xz' [157261076/157261076]

root@slack64:/usr/src# ls
linux@ linux-5.15.207/ linux-7.0.10.tar.xz linux-7.0.2/
root@slack64:/usr/src#
```

Rysunek 2: Pobranie źródeł najnowszej wersji jądra

Potem trzeba było archiwum rozpakować.

```
root@slack64:/usr/src# tar -xvpf linux-7.0.10.tar.xz
```

Rysunek 3: Rozpakowywanie plików

```

linux-7.0.10/virt/kvm/async_pf.h
linux-7.0.10/virt/kvm/binary_stats.c
linux-7.0.10/virt/kvm/coalesced_mmio.c
linux-7.0.10/virt/kvm/coalesced_mmio.h
linux-7.0.10/virt/kvm/dirty_ring.c
linux-7.0.10/virt/kvm/eventfd.c
linux-7.0.10/virt/kvm/guest_memfd.c
linux-7.0.10/virt/kvm/irqchip.c
linux-7.0.10/virt/kvm/kvm_main.c
linux-7.0.10/virt/kvm/kvm_mm.h
linux-7.0.10/virt/kvm/pfn_cache.c
linux-7.0.10/virt/kvm/vfio.c
linux-7.0.10/virt/kvm/vfio.h
linux-7.0.10/virt/lib/
linux-7.0.10/virt/lib/Kconfig
linux-7.0.10/virt/lib/Makefile
linux-7.0.10/virt/lib/irqbypass.c
root@slack64:/usr/src#
root@slack64:/usr/src#

```

Rysunek 4: Koniec rozpakowywania plików jądra

Potem wchodzę do rozpakowanego katalogu z plikami jądra oraz kopiuję obecną konfigurację jądra.

```

root@slack64:/usr/src# cd linux-7.0.10/
root@slack64:/usr/src/linux-7.0.10# zcat /proc/config.gz > .config

```

Rysunek 5: Kopiowanie obecnej konfiguracji jądra

```

root@slack64:/usr/src/linux-7.0.10# ls -la
razem 1548
drwxrwxr-x 26 root root 4096 maj 24 15:16 ./
drwxr-xr-x 5 root root 4096 maj 24 15:10 ../
-rw-rw-r-- 1 root root 24291 maj 23 13:09 .clang-format
-rw-rw-r-- 1 root root 374 maj 23 13:09 .clippy.toml
-rw-rw-r-- 1 root root 59 maj 23 13:09 .cocciconfig
-rw-rw-r-- 1 root root 239751 maj 24 15:16 .config
-rw-rw-r-- 1 root root 566 maj 23 13:09 .editorconfig
-rw-rw-r-- 1 root root 305 maj 23 13:09 .get_maintainer.ignore
-rw-rw-r-- 1 root root 105 maj 23 13:09 .gitattributes
-rw-rw-r-- 1 root root 2238 maj 23 13:09 .gitignore
-rw-rw-r-- 1 root root 54512 maj 23 13:09 .mailmap
-rw-rw-r-- 1 root root 66 maj 23 13:09 .pylintrc
-rw-rw-r-- 1 root root 369 maj 23 13:09 .rustfmt.toml
-rw-rw-r-- 1 root root 496 maj 23 13:09 COPYING
-rw-rw-r-- 1 root root 107789 maj 23 13:09 CREDITS
drwxrwxr-x 78 root root 4096 maj 23 13:09 Documentation/

```

Rysunek 6: Kopia konfiguracji jądra w pliku *.config*

Potem użyłem komendy *make localmodconfig* do tworzenia pełnej konfiguracji.

```
root@slack64:/usr/src/linux-7.0.10# make localmodconfig
```

Rysunek 7: Komenda *make localmodconfig*

A teraz najbardziej angażująca rozrywka, czyli trzymanie klawisza enter.

```
sysctl test driver (TEST_SYSCTL) [N/m/y/?] n
Compiler context-analysis warnings test (CONTEXT_ANALYSIS_TEST) [N/y/?] (NEW)
udelay test driver (TEST_UDELAY) [N/m/y/?] n
Test static keys (TEST_STATIC_KEYS) [N/m/?] n
kmod stress tester (TEST_KMOD) [N/m/?] n
module kallsyms find_symbol() test (TEST_KALLSYMS) [N/m/?] (NEW)
Test memcat_p() helper function (TEST_MEMCAT_P) [N/m/y/?] n
Test heap/page initialization (TEST_MEMINIT) [N/m/y/?] n
Test HMM (Heterogeneous Memory Management) (TEST_HMM) [N/m/y/?] n
Test freeing pages (TEST_FREE_PAGES) [N/m/y/?] n
Test floating point operations in kernel space (TEST_FPU) [N/m/y/?] n
Test clocksource watchdog in kernel space (TEST_CLOCKSOURCE_WATCHDOG) [N/m/y/?] n
Test module for correctness and stress of objpool (TEST_OBJPOOL) [N/m/?] (NEW)
#
# configuration written to .config
#
root@slack64:/usr/src/linux-7.0.10#
```

Rysunek 8: Koniec zabawy

Teraz dla wszystkich nowych opcji w jądrze ustawienie ich na wartości domyślne.

```
root@slack64:/usr/src/linux-7.0.10# make olddefconfig
```

Rysunek 9: Komenda *make olddefconfig*

```
root@slack64:/usr/src/linux-7.0.10# make olddefconfig
#
# No change to .config
#
root@slack64:/usr/src/linux-7.0.10#
```

Rysunek 10: Wynik komendy *make olddefconfig*

Na maszynie mam dostępne tylko 4 rdzenie. Stosując zalecaną metodę następne polecenia *make* będę wykonywał z opcją *-j6* stosując mnożnik *1.5x* dla ilości rdzeni. Teraz rozpoczynam kompilację jądra.

```
root@slack64:/usr/src/linux-7.0.10# make -j6 bzImage
```

Rysunek 11: Komenda *make bzImage*

Kompilacja trwała około 10 minut.

```
CC      arch/x86/boot/compressed/pgtable_64.o
CC      arch/x86/boot/compressed/sev.o
CC      arch/x86/boot/compressed/sev-handle-vc.o
CC      arch/x86/boot/compressed/acpi.o
CC      arch/x86/boot/compressed/mem.o
CC      arch/x86/boot/compressed/efi.o
CC      arch/x86/boot/compressed/misc.o
LZMA     arch/x86/boot/compressed/vmlinux.bin.lzma
MKPIGGY  arch/x86/boot/compressed/piggy.S
AS       arch/x86/boot/compressed/piggy.o
LD       arch/x86/boot/compressed/vmlinux
ZOFFSET  arch/x86/boot/zoffset.h
OBJCOPY  arch/x86/boot/vmlinux.bin
AS       arch/x86/boot/header.o
LD       arch/x86/boot/setup.elf
OBJCOPY  arch/x86/boot/setup.bin
BUILD   arch/x86/boot/bzImage
Kernel: arch/x86/boot/bzImage is ready (#1)
root@slack64:/usr/src/linux-7.0.10#
```

Rysunek 12: Koniec kompilacji *bzImage*

Teraz pora na moduły.

```
root@slack64:/usr/src/linux-7.0.10# make -j6 modules
```

Rysunek 13: Komenda *make modules*

Po 2-3 minutach jest gotowe.

```
LD [M] sound/core/snd-timer.ko
LD [M] sound/core/snd-pcm.ko
LD [M] sound/hda/core/snd-hda-core.ko
LD [M] sound/hda/core/snd-intel-dspcfg.ko
LD [M] sound/hda/common/snd-hda-codec.ko
LD [M] sound/hda/core/snd-intel-sdw-acpi.ko
LD [M] sound/hda/codecs/snd-hda-codec-generic.ko
LD [M] net/802/psnap.ko
LD [M] sound/hda/controllers/snd-hda-intel.ko
LD [M] net/802/garp.ko
LD [M] net/802/stp.ko
LD [M] net/ipv6/ipv6.ko
LD [M] net/802/mrp.ko
LD [M] net/8021q/8021q.ko
LD [M] net/wireless/cfg80211.ko
LD [M] net/llc/llc.ko
LD [M] net/rfkill/rfkill.ko
LD [M] virt/lib/irqbypass.ko
root@slack64:/usr/src/linux-7.0.10#
```

Rysunek 14: Koniec kompilacji modułów

Teraz pora na instalację.

```
root@slack64:/usr/src/linux-7.0.10# make -j6 modules_install
```

Rysunek 15: Komenda *make modules\_install*

Trwała dosłownie chwilę.

```

INSTALL /lib/modules/7.0.10/kernel/sound/core/snd-pcm.ko
INSTALL /lib/modules/7.0.10/kernel/sound/hda/core/snd-intel-dspcfg.ko
INSTALL /lib/modules/7.0.10/kernel/sound/hda/core/snd-hda-core.ko
INSTALL /lib/modules/7.0.10/kernel/sound/hda/core/snd-intel-sdw-acpi.ko
INSTALL /lib/modules/7.0.10/kernel/sound/hda/common/snd-hda-codec.ko
INSTALL /lib/modules/7.0.10/kernel/sound/hda/codecs/snd-hda-codec-generic.ko
INSTALL /lib/modules/7.0.10/kernel/net/802/psnap.ko
INSTALL /lib/modules/7.0.10/kernel/sound/hda/controllers/snd-hda-intel.ko
INSTALL /lib/modules/7.0.10/kernel/net/802/stp.ko
INSTALL /lib/modules/7.0.10/kernel/net/802/garp.ko
INSTALL /lib/modules/7.0.10/kernel/net/802/mrp.ko
INSTALL /lib/modules/7.0.10/kernel/net/ipv6/ipv6.ko
INSTALL /lib/modules/7.0.10/kernel/net/8021q/8021q.ko
INSTALL /lib/modules/7.0.10/kernel/net/wireless/cfg80211.ko
INSTALL /lib/modules/7.0.10/kernel/net/llc/llc.ko
INSTALL /lib/modules/7.0.10/kernel/net/rfkill/rfkill.ko
INSTALL /lib/modules/7.0.10/kernel/virt/lib/irqbypass.ko
DEPMOD /lib/modules/7.0.10
root@slack64:/usr/src/linux-7.0.10#

```

Rysunek 16: Koniec instalacji modułów

Pora na pliki nagłówkowe, nie są wymagane, ale będę je instalował.

```

root@slack64:/usr/src/linux-7.0.10# make -j6 headers_install

```

Rysunek 17: Instalacja plików nagłówkowych

Trwało to około 30 sekund.

```

HDRINST usr/include/asm/bpf_perf_event.h
HDRINST usr/include/asm/socket.h
HDRINST usr/include/asm/unistd_x32.h
HDRINST usr/include/asm/types.h
HDRINST usr/include/asm/param.h
HDRINST usr/include/asm/resource.h
HDRINST usr/include/asm/sockios.h
HDRINST usr/include/asm/termbits.h
HDRINST usr/include/asm/errno.h
HDRINST usr/include/asm/fcntl.h
HDRINST usr/include/asm/poll.h
HDRINST usr/include/asm/ioctls.h
HDRINST usr/include/asm/ipcbuf.h
HDRINST usr/include/asm/unistd_32.h
HDRINST usr/include/asm/ioctl.h
HDRINST usr/include/asm/termios.h
HDRINST usr/include/asm/unistd_64.h
INSTALL ./usr/include
root@slack64:/usr/src/linux-7.0.10#

```

Rysunek 18: Koniec instalacji plików nagłówkowych

Teraz kopiowanie potrzebnych plików do katalogu `/boot`.

```

root@slack64:/usr/src/linux-7.0.10# cp arch/x86_64/boot/bzImage /boot/vmlinuz-custom-7.0.10
root@slack64:/usr/src/linux-7.0.10# cp System.map /boot/System.map-custom-7.0.10
root@slack64:/usr/src/linux-7.0.10# cp .config /boot/config-custom-7.0.10
root@slack64:/usr/src/linux-7.0.10#

```

Rysunek 19: Kopiowanie plików

Przejdźcie do katalogu `/boot`.

```

root@slack64:/usr/src/linux-7.0.10# cd /boot
root@slack64:/boot# ls
README.initrd@          amd-ucode.img           efi/                    initrd.gz               tuxlogo.bmp             vmlinuz-generic-5.15.207
System.map@              config@                  elilo-ia32.efi*         inside.bmp              tuxlogo.dat             vmlinuz-huge@
System.map-custom-7.0.10 config-custom-7.0.10    elilo-x86_64.efi*      inside.dat              vmlinuz@                vmlinuz-huge-5.15.207
System.map-custom-7.0.2  config-custom-7.0.2    grub/                   onlyblue.bmp           vmlinuz-custom-7.0.10
System.map-generic-5.15.207 config-generic-5.15.207.x64 initrd-custom-7.0.2.gz  onlyblue.dat           vmlinuz-custom-7.0.2
System.map-huge-5.15.207 config-huge-5.15.207.x64 initrd-tree/            slack.bmp               vmlinuz-generic@
root@slack64:/boot#

```

Rysunek 20: Katalog `/boot`

Teraz trzeba usunąć stare dowiązanie `System.map` i stworzyć nowe o takiej samej nazwie.

```

root@slack64:/boot# rm System.map
root@slack64:/boot# ln -s System.map-custom-7.0.10 System.map
root@slack64:/boot# ls -l
razem 101041
lrwxrwxrwx 1 root root      38 kwi 13 16:54 README.initrd -> /usr/doc/mkinitrd-1.4.11/README.initrd
lrwxrwxrwx 1 root root      24 maj 24 16:42 System.map -> System.map-custom-7.0.10
-rw-r--r-- 1 root root 10659978 maj 24 16:35 System.map-custom-7.0.10
-rw-r--r-- 1 root root 10831123 maj 5 15:07 System.map-custom-7.0.2
-rw-r--r-- 1 root root 5145295 maj 15 22:14 System.map-generic-5.15.207
-rw-r--r-- 1 root root 6945911 maj 15 22:11 System.map-huge-5.15.207
-rw-r--r-- 1 root root 177152 wrz 12 2025 amd-ucode.img
lrwxrwxrwx 1 root root      24 maj 24 10:45 config -> config-huge-5.15.207.x64
-rw-r--r-- 1 root root 168551 maj 24 16:36 config-custom-7.0.10
-rw-r--r-- 1 root root 278039 maj 5 15:08 config-custom-7.0.2
-rw-r--r-- 1 root root 239782 maj 15 20:33 config-generic-5.15.207.x64
-rw-r--r-- 1 root root 239751 maj 15 20:33 config-huge-5.15.207.x64
drwxr-xr-x 3 root root     1024 sty 1 1970 efi/
-rwxr-xr-x 1 root root 216219 cze 12 2018 elilo-ia32.efi*
-rwxr-xr-x 1 root root 238531 cze 12 2018 elilo-x86_64.efi*

```

Rysunek 21: Tworzenie nowego dowiązania

Teraz trzeba wygenerować nowy RAM disk, tylko trzeba pamiętać, żeby w komendzie zmienić lokalację docelową zapisu pliku.



```
root@slack64:/boot# /usr/share/mkinitrd/mkinitrd_command_generator.sh -k 7.0.10
#
# mkinitrd_command_generator.sh revision 1.45
#
# This script will now make a recommendation about the command to use
# in case you require an initrd image to boot a kernel that does not
# have support for your storage or root filesystem built in
# (such as the Slackware 'generic' kernels').
# A suitable 'mkinitrd' command will be:
mkinitrd -c -k 7.0.10 -f ext4 -r /dev/vda1 -m ext4 -u -o /boot/initrd.gz
root@slack64:/boot#
```

Rysunek 22: Generowanie komendy do wygenerowania RAM disk

```
root@slack64:/boot# mkinitrd -c -k 7.0.10 -f ext4 -r /dev/vda1 -m ext4 -u -o /boot/initrd-custom-7.0.10.gz
51730 bloków
/boot/initrd-custom-7.0.10.gz created.
Be sure to run lilo again if you use it.
root@slack64:/boot#
```

Rysunek 23: Wygenerowany nowy RAM disk

Teraz trzeba wygenerować nowy wpis do grub, który i tak trzeba będzie poprawić.

```
root@slack64:/boot# grub-mkconfig -o /boot/grub/grub.cfg
```

Rysunek 24: Generowanie nowego *grub.cfg*

```

root@slack64:/boot# grub-mkconfig -o /boot/grub/grub.cfg
Generowanie pliku konfiguracyjnego gruba...
Znaleziono obraz Linuksa: /boot/vmlinuz-huge-5.15.207
Znaleziono obraz initrd: /boot/amd-ucode.img /boot/initrd.gz
Znaleziono obraz Linuksa: /boot/vmlinuz-huge
Znaleziono obraz initrd: /boot/amd-ucode.img /boot/initrd.gz
Znaleziono obraz Linuksa: /boot/vmlinuz-generic-5.15.207
Znaleziono obraz initrd: /boot/amd-ucode.img /boot/initrd.gz
Znaleziono obraz Linuksa: /boot/vmlinuz-generic
Znaleziono obraz initrd: /boot/amd-ucode.img /boot/initrd.gz
Znaleziono obraz Linuksa: /boot/vmlinuz-custom-7.0.10
Znaleziono obraz initrd: /boot/amd-ucode.img /boot/initrd.gz
Znaleziono obraz Linuksa: /boot/vmlinuz-custom-7.0.2
Znaleziono obraz initrd: /boot/amd-ucode.img /boot/initrd.gz
Uwaga: os-prober nie zostanie uruchomiony w celu wykrycia innych uruchamialnych partycji.
Systemy na nich nie zostaną dodane do konfiguracji rozruchowej GRUB-a.
Proszę sprawdzić dokumentację dotyczącą GRUB_DISABLE_OS_PROBER.
Dodawanie wpisu menu rozruchowego dla ustawień firmware'u UEFI...
gotowe
root@slack64:/boot# █

```

Rysunek 25: Konfiguracja

Trzeba było poprawić RAM disk, który został wybrany dla jądra 7.0.10, ale również dla 7.0.2 (tego które było kompilowane na zajęciach).

```

        echo 'Ładowanie systemu Linux 7.0.10...'
        linux /boot/vmlinuz-custom-7.0.10 root=UUID=16ce0fca-25c9-4de3-81aa-f801b0e5d5ee ro
        echo 'Ładowanie początkowego ramdysku...'
        initrd /boot/amd-ucode.img /boot/initrd-custom-7.0.10.gz
    }
    menuentry 'Slackware-15.0 GNU/Linux, z systemem Linux 7.0.10 (tryb odzyskiwania)' --class slackware_15_0 --class gnu-linux --class gnu --class os $menuentry_id_option 'gnulinux-7.0.10-recovery-16ce0fca-25c9-4de3-81aa-f801b0e5d5ee' {
        load_video
        insmod gzio
        insmod part_gpt
        insmod ext2
        search --no-floppy --fs-uuid --set=root 16ce0fca-25c9-4de3-81aa-f801b0e5d5ee
        echo 'Ładowanie systemu Linux 7.0.10...'
        linux /boot/vmlinuz-custom-7.0.10 root=UUID=16ce0fca-25c9-4de3-81aa-f801b0e5d5ee ro single
        echo 'Ładowanie początkowego ramdysku...'
        initrd /boot/amd-ucode.img /boot/initrd-custom-7.0.10.gz
    }
    menuentry 'Slackware-15.0 GNU/Linux, z systemem Linux 7.0.2' --class slackware_15_0 --class gnu-linux --class gnu --class os $menuentry_id_option 'gnulinux-7.0.2-advanced-16ce0fca-25c9-4de3-81aa-f801b0e5d5ee' {
        load_video

```

Rysunek 26: Plik *grub.cfg*

Teraz czas na konfigurację elilo. Zaczynając od kopiowania plików do katalogu */boot/efi/EFI/Slackware*.

```

root@slack64:/boot# cp /boot/vmlinuz-custom-7.0.10 /boot/efi/EFI/Slackware/
root@slack64:/boot# cp /boot/initrd-custom-7.0.10.gz /boot/efi/EFI/Slackware/
root@slack64:/boot# █

```

Rysunek 27: Kopiowanie plików do */boot/efi/EFI/Slackware*

Po skopiowaniu plików trzeba zmodyfikować plik */boot/efi/EFI/Slackware/elilo.conf* i dodać nowy wpis.

```

prompt
delay=15
timeout=30
default=custom-kernel-7.0.2
#
image=vmlinuz
    label="Slackware 15.0"
    initrd=initrd.gz
    read-only
    append="root=/dev/vda1 vga=791 ro"

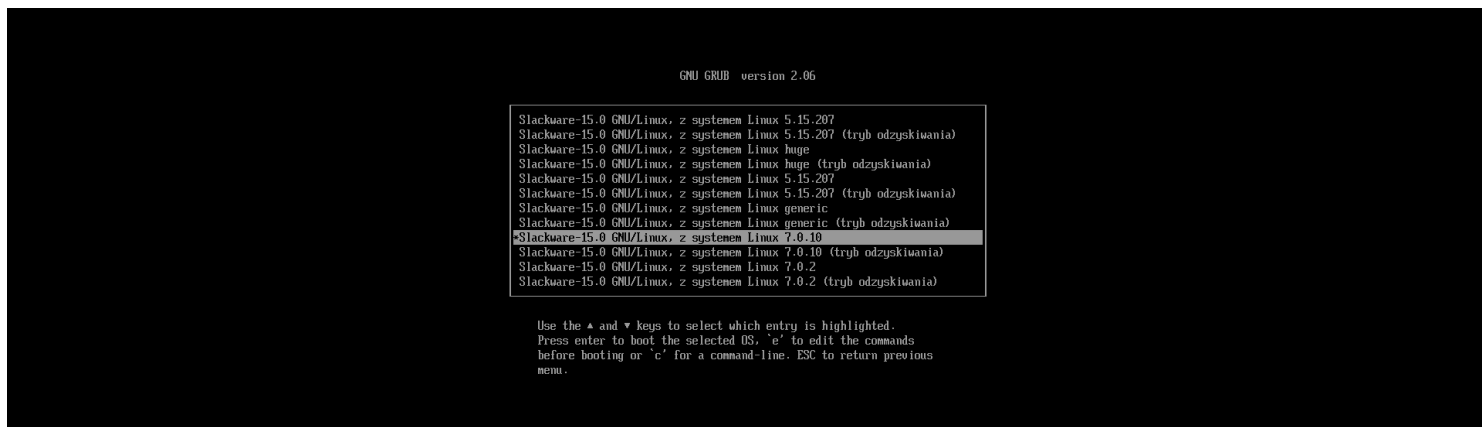
image=vmlinuz-custom-7.0.2
    label=custom-kernel-7.0.2
    initrd=initrd-custom-7.0.2.gz
    read-only
    append="root=/dev/vda1 vga=791 ro"

image=vmlinuz-custom-7.0.10
    label=custom-kernel-7.0.10-oldmethod
    initrd=initrd-custom-7.0.10.gz
    read-only
    append="root=/dev/vda1 vga=791 ro"

```

Rysunek 28: Nowy wpis w *elilo.conf*

Teraz trzeba zapisać i zrestartować system.



Rysunek 29: Wpis w grub

Udało się przy zalogowaniu widzieć, że aktywne jest jądro 7.0.10.

```

Last login: Sun May 24 15:04:44 2026 from 192.168.122.1
Linux 7.0.10.
root@slack64:~#

```

Rysunek 30: Nowe jądro systemu

## 3 Nowa metoda

### 3.1 Przebieg kompilacji

Dla jednolitości, wróć na wersję jądra 5.15.207. Tutaj pojawia się problem. Zaczęło się kończyć miejsce na dysku. Będę wykonywał tyle ile mi się uda, może do końca. Najpierw zrobię kopię starego pliku konfiguracyjnego.

```
root@slack64:/usr/src/linux-7.0.10# mv .config config_old
root@slack64:/usr/src/linux-7.0.10#
```

Rysunek 31: Kopiowanie pliku konfiguracyjnego

Teraz trzeba pobrać znowu obecną konfigurację.

```
root@slack64:/usr/src/linux-7.0.10# zcat /proc/config.gz > .config
root@slack64:/usr/src/linux-7.0.10#
```

Rysunek 32: Pobranie znowu konfiguracji

Użycie metody streamlineconfig do tworzenia pliku konfiguracyjnego.

```
root@slack64:/usr/src/linux-7.0.10# scripts/kconfig/streamline_config.pl > .config_new
using config: '.config'
sha1_ssse3 config not found!
crc32_pclmul config not found!
ledtrig_audio config not found!
sha256_ssse3 config not found!
crct10dif_pclmul config not found!
module fb_sys_fops did not have configs CONFIG_FB_SYSMEM_FOPS
root@slack64:/usr/src/linux-7.0.10#
```

Rysunek 33: Streamline config

Usunięcie pliku konfiguracyjnego i zastąpieniem go tym wygenerowanym przez komendę.

```
root@slack64:/usr/src/linux-7.0.10# rm .config
root@slack64:/usr/src/linux-7.0.10# mv .config_new .config
root@slack64:/usr/src/linux-7.0.10#
```

Rysunek 34: Usunięcie pliku

Tutaj nie wiedziałem, chyba w tej metodzie nie trzeba robić tego kroku, ale dla pewności go zrobiłem.

```
root@slack64:/usr/src/linux-7.0.10# make olddefconfig
```

Rysunek 35: Komenda *make olddefconfig*

```

root@slack64:/usr/src/linux-7.0.10# make olddefconfig
HOSTCC scripts/kconfig/conf.o
HOSTCC scripts/kconfig/confdata.o
HOSTCC scripts/kconfig/expr.o
LEX scripts/kconfig/lexer.lex.c
YACC scripts/kconfig/parser.tab.[ch]
HOSTCC scripts/kconfig/lexer.lex.o
HOSTCC scripts/kconfig/menu.o
HOSTCC scripts/kconfig/parser.tab.o
HOSTCC scripts/kconfig/preprocess.o
HOSTCC scripts/kconfig/symbol.o
HOSTCC scripts/kconfig/util.o
HOSTLD scripts/kconfig/conf
.config:860:warning: symbol value '0' invalid for BASE_SMALL
#
# configuration written to .config
#
root@slack64:/usr/src/linux-7.0.10# 

```

Rysunek 36: Wynik komendy

```

root@slack64:/usr/src/linux-7.0.10# make -j6 bzImage

```

Rysunek 37: Kompilacja jądra

Zaskakująco szybko poszło, w porównaniu z poprzednim razem.

```

MM      System.map
SORTTAB vmlinux.unstripped
OBJCOPY vmlinux
GEN      modules.builtin.modinfo
GEN      modules.builtin
CC       arch/x86/boot/version.o
VOFFSET  arch/x86/boot/compressed/..voffset.h
OBJCOPY  arch/x86/boot/compressed/vmlinux.bin
RELOCS   arch/x86/boot/compressed/vmlinux.relocs
CC       arch/x86/boot/compressed/kaslr.o
LZMA     arch/x86/boot/compressed/vmlinux.bin.lzma
CC       arch/x86/boot/compressed/misc.o
MKPIGGY  arch/x86/boot/compressed/piggy.S
AS       arch/x86/boot/compressed/piggy.o
LD       arch/x86/boot/compressed/vmlinux
ZOFFSET  arch/x86/boot/zoffset.h
OBJCOPY  arch/x86/boot/vmlinux.bin
AS       arch/x86/boot/header.o
LD       arch/x86/boot/setup.elf
OBJCOPY  arch/x86/boot/setup.bin
BUILD    arch/x86/boot/bzImage
Kernel: arch/x86/boot/bzImage is ready (#3)
root@slack64:/usr/src/linux-7.0.10# 

```

Rysunek 38: Koniec kompilacji

```
root@slack64:/usr/src/linux-7.0.10# make -j6 modules
```

Rysunek 39: Tworzenie modułów

```
LD [M] drivers/powercap/intel_rapl_msr.ko
LD [M] sound/soundcore.ko
LD [M] sound/core/snd.ko
LD [M] sound/core/snd-hwdep.ko
LD [M] sound/core/snd-timer.ko
LD [M] sound/core/snd-pcm.ko
LD [M] sound/hda/core/snd-intel-dspcfg.ko
LD [M] sound/hda/core/snd-hda-core.ko
LD [M] sound/hda/core/snd-intel-sdw-acpi.ko
LD [M] sound/hda/common/snd-hda-codec.ko
LD [M] sound/hda/codecs/snd-hda-codec-generic.ko
LD [M] sound/hda/controllers/snd-hda-intel.ko
LD [M] net/802/stp.ko
LD [M] net/802/psnap.ko
LD [M] net/802/mrp.ko
LD [M] net/802/garp.ko
LD [M] net/ipv6/ipv6.ko
LD [M] net/8021q/8021q.ko
LD [M] net/wireless/cfg80211.ko
LD [M] net/llc/llc.ko
LD [M] net/rfkill/rfkill.ko
LD [M] virt/libirqbypass.ko
root@slack64:/usr/src/linux-7.0.10#
```

Rysunek 40: Koniec kompilacji modułów

```
root@slack64:/usr/src/linux-7.0.10# make -j6 modules_install
```

Rysunek 41: Instalacja modułów

Tutaj nie udało mi się zrobić w czasie zdjęcia wyniku.

```
root@slack64:/usr/src/linux-7.0.10# make -j6 headers_install
```

Rysunek 42: Instalacja plików nagłówkowych

```
HDRINST usr/include/asm/msgbuf.h
HDRINST usr/include/asm/mtrr.h
HDRINST usr/include/asm/mce.h
HDRINST usr/include/asm/kvm_perf.h
HDRINST usr/include/asm/bpf_perf_event.h
HDRINST usr/include/asm/socket.h
HDRINST usr/include/asm/unistd_x32.h
HDRINST usr/include/asm/types.h
HDRINST usr/include/asm/param.h
HDRINST usr/include/asm/resource.h
HDRINST usr/include/asm/sockios.h
HDRINST usr/include/asm/termbits.h
HDRINST usr/include/asm/errno.h
HDRINST usr/include/asm/fcntl.h
HDRINST usr/include/asm/poll.h
HDRINST usr/include/asm/ioctls.h
HDRINST usr/include/asm/ipcbuf.h
HDRINST usr/include/asm/unistd_32.h
HDRINST usr/include/asm/ioctl.h
HDRINST usr/include/asm/termios.h
HDRINST usr/include/asm/unistd_64.h
INSTALL ./usr/include
root@slack64:/usr/src/linux-7.0.10#
```

Rysunek 43: Koniec instalacji

Teraz trzeba przekopiować pliki do katalogu */boot*.

```
root@slack64:/usr/src/linux-7.0.10# cp arch/x86_64/boot/bzImage /boot/vmlinuz-custom-new-7.0.10
root@slack64:/usr/src/linux-7.0.10# cp System.map /boot/System.map-custom-new-7.0.10
root@slack64:/usr/src/linux-7.0.10# cp .config /boot/config-custom-new-7.0.10
root@slack64:/usr/src/linux-7.0.10#
```

Rysunek 44: Kopiowanie plików

Teraz w katalogu */boot* trzeba usunąć dowiązanie, stworzyć nowe, oraz wygenerować nowy RAM disk. Tutaj nie wiem czy koniecznie trzeba, czy może wystarczy poprzedni? Nie sprawdzałem i nie ryzykowałem.



```

root@slack64:/boot# rm System.map
root@slack64:/boot# ln -s System.map-custom-new-7.0.10 System.map
root@slack64:/boot# ls
README.initrd@      amd-ucode.img      efi/               initrd.gz          tuxlogo.dat        vmlinuz-huge@
System.map@          config@            elilo-ia32.efi*    inside.bmp         vmlinuz@           vmlinuz-huge-5.15.207
System.map-custom-7.0.10  config-custom-7.0.10  elilo-x86_64.efi*  inside.dat         vmlinuz-custom-7.0.10
System.map-custom-7.0.2  config-custom-7.0.2  grub/              onlyblue.bmp       vmlinuz-custom-7.0.2
System.map-custom-new-7.0.10  config-custom-new-7.0.10  initrd-custom-7.0.10.gz  onlyblue.dat       vmlinuz-custom-new-7.0.10
System.map-generic-5.15.207  config-generic-5.15.207.x64  initrd-custom-7.0.2.gz  slack.bmp          vmlinuz-generic@
System.map-huge-5.15.207    config-huge-5.15.207.x64  initrd-tree/        tuxlogo.bmp       vmlinuz-generic-5.15.207
root@slack64:/boot#

```

Rysunek 45: Plik *System.map*

```

root@slack64:/boot# /usr/share/mkinitrd/mkinitrd_command_generator.sh -k 7.0.10
#
# mkinitrd_command_generator.sh revision 1.45
#
# This script will now make a recommendation about the command to use
# in case you require an initrd image to boot a kernel that does not
# have support for your storage or root filesystem built in
# (such as the Slackware 'generic' kernels').
# A suitable 'mkinitrd' command will be:
mkinitrd -c -k 7.0.10 -f ext4 -r /dev/vda1 -m ext4 -u -o /boot/initrd.gz
root@slack64:/boot#

```

Rysunek 46: Polecenie do wygenerowania RAM disk

Teraz generowanie RAM disk. Trzeba zmienić nazwę docelową.

```

root@slack64:/boot# mkinitrd -c -k 7.0.10 -f ext4 -r /dev/vda1 -m ext4 -u -o /boot/initrd-custom-new-7.0.10.gz
51730 bloków
/boot/initrd-custom-new-7.0.10.gz created.
Be sure to run lilo again if you use it.
root@slack64:/boot#

```

Rysunek 47: Stworzony nowy RAM disk

```

root@slack64:/boot# grub-mkconfig -o /boot/grub/grub.cfg
Generowanie pliku konfiguracyjnego gruba...
Znaleziono obraz Linuksa: /boot/vmlinuz-huge-5.15.207
Znaleziono obraz initrd: /boot/amd-ucode.img /boot/initrd.gz
Znaleziono obraz Linuksa: /boot/vmlinuz-huge
Znaleziono obraz initrd: /boot/amd-ucode.img /boot/initrd.gz
Znaleziono obraz Linuksa: /boot/vmlinuz-generic-5.15.207
Znaleziono obraz initrd: /boot/amd-ucode.img /boot/initrd.gz
Znaleziono obraz Linuksa: /boot/vmlinuz-generic
Znaleziono obraz initrd: /boot/amd-ucode.img /boot/initrd.gz
Znaleziono obraz Linuksa: /boot/vmlinuz-custom-new-7.0.10
Znaleziono obraz initrd: /boot/amd-ucode.img /boot/initrd.gz
Znaleziono obraz Linuksa: /boot/vmlinuz-custom-7.0.10
Znaleziono obraz initrd: /boot/amd-ucode.img /boot/initrd.gz
Znaleziono obraz Linuksa: /boot/vmlinuz-custom-7.0.2
Znaleziono obraz initrd: /boot/amd-ucode.img /boot/initrd.gz
Uwaga: os-prober nie zostanie uruchomiony w celu wykrycia innych uruchamialnych partycji.
Systemy na nich nie zostaną dodane do konfiguracji rozruchowej GRUB-a.
Proszę sprawdzić dokumentację dotyczącą GRUB DISABLE_OS_PROBER.
Dodawanie wpisu menu rozruchowego dla ustawień firmware'u UEFI...
gotowe
root@slack64:/boot# █

```

Rysunek 48: Generowanie wpisów Grub

Które trzeba znowu zedytować. Teraz czas na elilo.

```

root@slack64:/boot# cp /boot/vmlinuz-custom-new-7.0.10 /boot/efi/EFI/Slackware/
root@slack64:/boot# cp /boot/initrd-custom-new-7.0.10.gz /boot/efi/EFI/Slackware/
root@slack64:/boot# █

```

Rysunek 49: Kopiowanie plików

I również trzeba dodać wpis do *elilo.conf* analogicznie do poprzedniego punktu w starej metodzie.



Rysunek 50: Menu grub

Jak widać są dwa wpisy na jądro 7.0.10. System startuje i wszystko działa.

## 4 Wnioski

Nie było żadnych problemów przy instalacji jądra starą metodą (`localmodconfig`), można było robić wszystko analogicznie jak na zajęciach. Ułatwiałoby to, że wszystkie komendy były praktycznie podane w `.bash_history`.

Były lekkie problemy z miejscem na dysku, ale w końcu się udało. Wiem, że na zajęciach generowaliśmy plik konfiguracyjny, ale o tym zapomniałem i zrobiłem to dla nowszej wersji jądra. Folder źródłowy został skompresowany i wstawiony na Github, ale w częściach, ze względu na limit na wielkość pojedynczego pliku.